

1.2. Классификация алгоритмов

Алгоритмы компьютерной графики можно разделить на два уровня: нижний и верхний. Алгоритмы нижнего уровня предназначены для реализации графических примитивов (линий, окружностей, заливки, штриховок и т.д.). такие алгоритмы воспроизведены в графических библиотеках языков высокого уровня или реализованы аппаратным способом в графических процессорах рабочих станций и графических систем. Среди алгоритмов нижнего уровня можно выделить следующие группы:

- ❖ простейшие в смысле используемых математических моделей и отличающиеся простотой реализации. Как правило, такие алгоритмы не являются оптимальными по объему вычислений или требуемым ресурсам памяти;
- ❖ алгоритмы, использующие достаточно сложные математические методы, и некоторые эвристические алгоритмы, отличающиеся большей эффективностью;
- ❖ аппаратно-реализуемые, к которым можно отнести алгоритмы, допускающие распараллеливание, рекурсивные алгоритмы и алгоритмы, реализуемые в командах процессора;
- ❖ алгоритмы специального назначения, например, устранения лестничного эффекта в изображении.

К алгоритмам верхнего уровня относятся в первую очередь алгоритмы удаления невидимых линий и поверхностей. Задача удаления невидимых линий и поверхностей продолжает оставаться центральной в машинной графике. От эффективности алгоритмов решения этой задачи зависят качество и скорость построения трехмерного изображения.

К задаче удаления невидимых линий и поверхностей примыкает задача построения полутоновых реалистичных изображений. Решение этой задачи требует учета явлений, связанных с количеством и характером источников света, учета свойств поверхности тела (прозрачность, преломление, отражение света).

При этом не следует забывать, что построение объектов в алгоритмах верхнего уровня обеспечивается примитивами на основе алгоритмов нижнего уровня, поэтому нельзя игнорировать проблему выбора и разработки эффективных алгоритмов нижнего уровня.

Для разных областей применения машинной графики на первый план могут выдвигаться разные свойства алгоритмов. Для научной графики большое значение имеет универсальность алгоритма, быстродействие отходит на второй план. Для систем моделирования, в том числе движущихся объектов, быстродействие становится главным критерием.

1.3. Основы построения растровых изображений

Пиксели на экране монитора (растр) имеют конечные физические размеры и их количество ограничено. Растр представляет собой конечное

дискретное множество точек. Поэтому воспроизведение изображения на экране дисплея (или на другом растровом устройстве) требует выполнения определенных процедур аппроксимации и связано с частичной потерей информации. Отображение любого объекта на целочисленную решетку называется разложением его в растр или растровым представлением.

Рассмотрим для примера задачу растрового представления прямой линии. Можно попытаться воспользоваться уравнением прямой линии $y = ax + b$, задавая аргументу x последовательность целых значений. Результаты вычисления значений переменной y надо округлять до целых. В итоге, вместо непрерывной прямой линии мы получим ступенчатую линию или даже набор изолированных точек.

Аналогичная ситуация имеет место для окружности или эллипса. Уравнение окружности выглядит следующим образом:

$$\begin{aligned}x &= x_c + r \times \cos(\varphi), \\ y &= y_c + r \times \sin(\varphi),\end{aligned}\tag{1}$$

где (x_c, y_c) – координаты центра окружности, r – радиус, φ – угол для текущей точки (x, y) .

Можно строить окружность по уравнению (1), задав определенный шаг по углу ($\varphi = 0, \dots, 360^\circ$). Однако если шаг будет слишком мал, окружность за счет округления координат станет неровной, и некоторые точки высветятся несколько раз. Обычно шаг по углу выбирается равным $\frac{1}{r}$ радиан. Чаще всего шаг изменения угла должен быть переменным для того, чтобы избежать разрывов или отсутствия изменения координат.

Для ускорения построения окружности можно использовать ее симметрию. В этом случае рассчитываются координаты точек только 1/8 части окружности – для сегмента от 0 до 45 градусов. Чтобы избежать выполнения сложных операций вычисления синуса и косинуса, можно воспользоваться рекуррентными формулами, выражающими координаты следующей точки окружности через координаты предыдущей точки:

$$\begin{aligned}x_2 &= x_c + (x_1 - x_c) \times cs + (y_1 - y_c) \times sn, \\ y_2 &= y_c + (y_1 - y_c) \times cs - (x_1 - x_c) \times sn,\end{aligned}\tag{2}$$

где $sn = \sin(\Delta\varphi)$, $cs = \cos(\Delta\varphi)$ и $\Delta\varphi$ – шаг по углу. Для рекуррентной формулы (2) начальная точка вычисляется по следующей формуле: где $x_1 = x_c, y_1 = y_c + r$.